国立大学法人
電気通信大学
The University of Electro-Communications

Pham Laboratory
Integrated circuit design laboratory

「Course」
EDHOC on a Rocket RV64 SoC

「Lesson 5」 Secure file transfer and results

Khai-Duy Nguyen

2026/12

Today’s lesson purpose:

- Use the key from the handshake: an encrypted channel that streams a file of any size from one board's SD card to the other's.
- Make the radio link and the SD card reliable — and learn from the bugs we hit.
- Measure where the time goes, make it faster (radio, then a 3× faster wired link), and see how each question behind the project was answered.

Code: fpga/sw/export/common (edhoc_app_data, edhoc_rf_link, sd_card, fat32, edhoc_file_xfer)

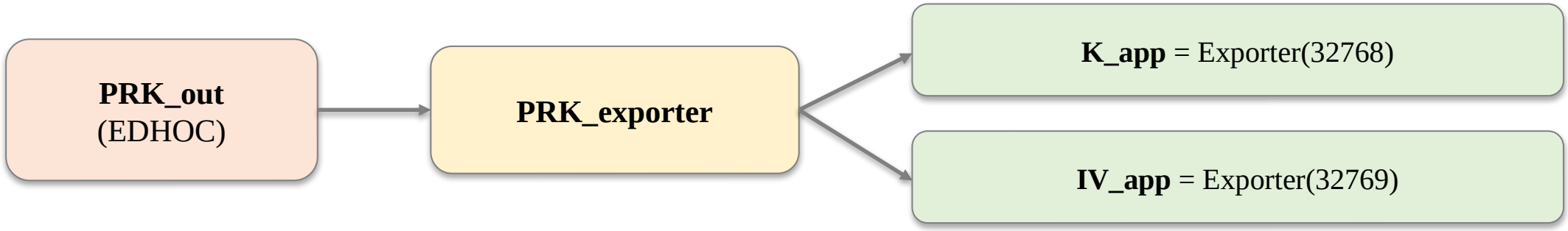
Outline

1. Application Channel
2. Radio Link
3. SD Card & FAT32
4. End-to-End Transfer
5. Making It Faster
6. Questions Behind the Project
7. Practice

Outline

1. **Application Channel**
2. Radio Link
3. SD Card & FAT32
4. End-to-End Transfer
5. Making It Faster
6. Questions Behind the Project
7. Practice

1. Application Channel (1/4) Keys for the application



- Same suite, same keys on both boards — nothing secret is sent.
- The AEAD is the suite's **application AEAD**: AES-CCM-16-64-128, AES-128/256-GCM or ChaCha20/Poly1305; key, IV and tag sizes are read at run time, so one code path serves every suite.
- The file demo runs with AuthKEM TBD3: AES-256-GCM, 32-byte key.

1. Application Channel (2/4) One record

Part	Content	Sent?
header (5 B)	direction (0 = A→B, 1 = B→A) ‖ 32-bit sequence number	no — both sides know it
nonce	IV_app XOR header (in the last 5 bytes)	no
ciphertext	AEAD(K_app, nonce, aad = header, plaintext ≤ 206 B)	yes
tag	8 or 16 bytes	yes

common/src/edhoc_app_data.c

```
static void mk_nonce(const struct app_chan *ch, uint8_t *nonce, const uint8_t *hdr)
{
    uint32_t off = ch->iv_len - APP_HDR_LEN;
    memcpy(nonce, ch->base_iv, ch->iv_len);
    for (uint32_t i = 0; i < APP_HDR_LEN; i++)
        nonce[off + i] ^= hdr[i];
}
```

- The two directions never share a nonce.
- A lost, repeated or reordered record fails its tag.

1. Application Channel (3/4) Using it from your code

```
common/include/edhoc_app_data.h
struct app_chan ch;
app_data_init(&ch, suite, &prk_out, is_initiator);    // keys from PRK_out

app_data_send(&ch, buf, len, tx, sock);               // len <= APP_PT_MAX (206)
app_data_recv(&ch, buf, sizeof(buf), &got, rx, sock); // decrypt + verify
```

- *tx* / *rx* are the same transport functions as the handshake (radio or UART).
- Records are at most 206 bytes: 206 + 16-byte tag fills exactly two radio fragments (section 2).
- Every send and receive also adds to an encrypt / decrypt cycle counter — the `PERF encrypt` / `PERF decrypt` lines.

1. Application Channel (4/4) Demo traffic

```
initiator, TBD3
APP: channel up (aead=3 key=32 iv=12 tag=16)
APP: sent    'REQ 0 FROM INITIATOR'
APP: got     'ECHO[REQ 0 FROM INITIATOR]'
APP: sent    'REQ 1 FROM INITIATOR'
APP: got     'ECHO[REQ 1 FROM INITIATOR]'
...
PERF encrypt      89688 cyc 0.001195 s 60 bytes 50209 B/s
APP DATA OK
```

- After every handshake: three requests and echoes both ways prove both keys and both directions.
- Then, depending on the build:
  - `APP_FILE_BYTES=n` — stream n generated bytes (checked by regenerating them)
  - `-DAPP_SD_XFER` — send SEND.BIN card to card (section 4)

Outline

1. Application Channel
2. **Radio Link**
3. SD Card & FAT32
4. End-to-End Transfer
5. Making It Faster
6. Questions Behind the Project
7. Practice

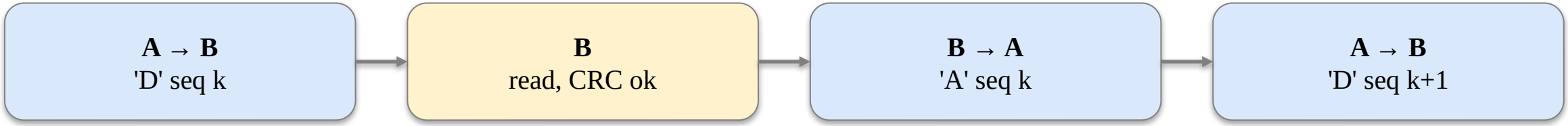
2. Radio Link (1/8) The MRF24J40 and its limits

Property	Value	Consequence
Standard	IEEE 802.15.4, 2.4 GHz, channel 11	shared band with Wi-Fi
Bit rate	250 kbit/s (32 µs per byte)	≈ 4 ms on air for a full frame
Frame	127 B → 116 B payload after the 802.15.4 header	a record needs 2 frames
Receive buffer	one frame	the sender must wait until the receiver read it
Host interface	SPI at 4.7 MHz over PMOD wires	bit errors happen (next slides)

- Hardware ACK: the radio confirms the frame reached the peer's receive buffer.
- It does **not** confirm the peer's software read it — that needs our own ACK.

2. Radio Link (2/8) Fragments and stop-and-wait

```
common/include/edhoc_rf_link.h
frame payload = [type][seq][flags][data ... up to 111 B][crc16]
  type  'D' data fragment   flags bit0 = first, bit1 = last of the message
        'A' acknowledgement of fragment seq
```



- The next fragment goes out only after the 'A' for the previous one.
- No 'A' within 30 ms → send the same fragment again; the receiver re-acks a repeat and drops it.
- Simple and safe with a one-frame receive buffer; the price is one round trip per fragment.

2. Radio Link (3/8) One radio frame, byte by byte

Bytes	Field	Written by
2	frame control: data frame, ACK request, PAN ID compression, short addresses	mrf24j40.c
1	802.15.4 sequence number	mrf24j40.c
2	PAN ID 0xABCD	mrf24j40.c
2 + 2	destination / source short address	mrf24j40.c
3	link header: type 'D' or 'A', seq, flags (bit 0 FIRST, bit 1 LAST)	edhoc_rf_link.c
≤ 111	data	edhoc_rf_link.c
2	CRC-16/CCITT over link header + data	edhoc_rf_link.c
2	FCS (802.15.4 CRC)	radio chip

- 9 + 3 + 111 + 2 + 2 = **127 bytes**, the largest 802.15.4 frame.
- Two checksums: the FCS covers the air, our CRC-16 also covers the SPI wires on both sides (Problem 1). Our 'A' frame is 16 bytes: header, link header, CRC, FCS.

2. Radio Link (4/8) Splitting and rebuilding a message

```
common/src/edhoc_rf_link.c (shortened)
int32_t rf_link_recv(uint8_t *data, uint32_t cap)
{
    len = recv_frag(f);                    // next 'D' in order, already ACKed
    if (!(f[2] & F_FIRST)) return -2;      // must start a message
    for (;;) {
        copy f[3 .. len) to data + got;    // -2 if it would pass cap
        if (f[2] & F_LAST) return got;     // message complete
        len = recv_frag(f);
        if (f[2] & F_FIRST) return -2;     // a new message inside this one
    }
}
```

Message	Bytes	Fragments
app record: 206 B + 16-B tag	222	111 FIRST, 111 LAST
length / CRC record: 4 B + tag	20	1 (FIRST + LAST)
RFC 9529 message_2	45	1
TBD3 message_4	1,604	14 × 111 + 50 = 15

- seq is 8 bits and wraps at 256: safe, because only one fragment is ever unconfirmed.
- A repeat of the fragment accepted last (its 'A' was lost) is acknowledged again and dropped, so data is never delivered twice.
- The first fragment of the peer's next message also confirms our last one (implicit ACK, *take_reply()*).

2. **Radio Link** (5/8) Problem 1: corrupted bytes

Symptom	the first 1 MiB stream failed after ~80 s with an AEAD tag error (113) — no SD card involved
Observation	the radio had acknowledged the frame, so its FCS was valid; the bytes the receiver decrypted were not the bytes sent
Cause	a bit flips now and then on the SPI wires between CPU and radio. The radio computes its FCS after the flip, so it cannot see it
Fix	our own CRC-16 per frame (seal() / poll_frame() in edhoc_rf_link.c), computed before SPI on the way out and checked after SPI on the way in; a bad frame gets no 'A' and is sent again
Check it	the RF: line at the end of every run counts crc_err. A long transfer with crc_err > 0 that still ends in XFER RECV … OK shows the CRC catching and recovering
Lesson	check data end to end yourself; a link layer's checksum covers only its own part of the path

2. Radio Link (6/8) Problem 2: a radio goes deaf

Symptom	with the CRC in place, the stream then stopped for good after ~420 KB
Observation	the sender repeated one fragment 9,140 times; the receiver received nothing at all
Cause	inference, not observed directly: a flipped SPI write left a radio control register wrong (e.g. the channel). A data CRC cannot repair that
Fix	a watchdog (heal_if_stuck): no progress for 2 s → reset and re-program the radio, keeping all sequence numbers. It must run in every waiting loop — the first version missed the transmit retry loop
Check it	the RF: line counts reinits. The responder shows a few while it waits for the initiator to be loaded (2 s each) — harmless; in a long transfer, reinits > 0 with a completed transfer shows recovery
Lesson	recover from stuck hardware instead of trusting it: every wait needs a way out

2. Radio Link (7/8) Problem 3: both sides repeating

Symptom	with DEBUG=1 builds the pair hung right after the handshake; also with a normal build when the initiator paused 0.5 s
Observation	read over JTAG: both boards in the transmit loop, each repeating one frame ~40 times a second, never acknowledged (TX status 0xC1); the watchdog did not help
Cause	a radio holding an unread frame in its one-frame FIFO was seen to acknowledge nothing — and neither side read its FIFO while repeating. Same root: a board that had finished left its peer repeating the last frame forever
Fix	take_reply() reads the FIFO between attempts (the peer's 'A' or next message ends the repeat); a finished board keeps answering (transport_idle())
Check it	comment out the take_reply() call in transmit(), build suite 2 method 3 with DEBUG=1, run the pair: it stops in the first echo
Restore	from fpga/sw/export: git checkout common/src/edhoc_rf_link.c, rebuild both images, rerun the same command: PASS. Compare the two logs at the point where the broken build stopped
Lesson	a link layer must keep listening while it waits

2. Radio Link (8/8) What each fix changed

Link version	**1 MiB radio stream (TBD3)**
no CRC, no watchdog	AEAD error after ~80 s (Problem 1)
+ CRC per frame	stalls after ~420 KB: deaf radio (Problem 2)
+ 2 s watchdog	completes (~135 s, 0 bad bytes) — hangs if both ends repeat
+ FIFO read while repeating	completes (~99 s), also with a slow peer (Problem 3)

- The same link carries the handshakes of Lessons 3 and 4, so every pair uses these fixes.
- Lesson: on real wires, **check data end to end and recover from stuck hardware** — do not trust the link layer of the chip alone.

Outline

1. Application Channel
2. Radio Link
3. **SD Card & FAT32**
4. End-to-End Transfer
5. Making It Faster
6. Questions Behind the Project
7. Practice

3. SD Card & FAT32 (1/8) What is on the card

Sectors (LBA)	Content	Written by
0	MBR partition table	mksd.sh / sd_prep
2048 … 67583 (partition 1, 32 MiB, raw)	firmware image for SD boot	sd_boot.sh
67584 … end (partition 2, FAT32)	SEND.BIN (board A), RECV.BIN (board B)	your PC / the board

- **End-user flow:** copy any file onto the card as SEND.BIN with a PC, put the card in board A, run — board B writes RECV.BIN on its card.
- FAT32, so any PC reads and writes the files; the boot area stays raw.
- Files up to 4 GiB (FAT32 limit); RAM use does not depend on the size.

3. SD Card & FAT32 (2/8) Preparing a card — with or without a card reader

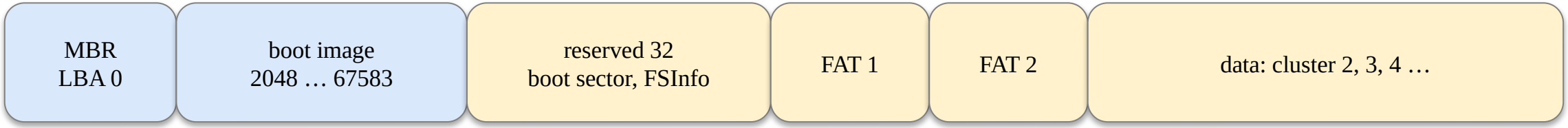
```
Run this — on a PC with a card reader
sudo fpga/sw/export/tools/mksd.sh /dev/sdX   # erases the card: MBR + 32 MiB boot + FAT32
# then copy your file as SEND.BIN onto the FAT32 partition (label EDHOC)
```

```
Run this — no card reader: from the board, over JTAG
cd fpga/sw/export/tests
make sd_prep EXTRA_CFLAGS="-DSD_PREP_FORMAT -DSD_PREP_SEND=1048576"   # format + 1 MiB test file
../common/board/run_one.sh build/sd_prep.elf 150
```

```
card: 61067264 sectors (29818 MiB)
formatting...
format OK
SEND.BIN: 1048576 bytes, CRC cffbfb2e
SD PREP OK
```

- *sd_prep* writes the same layout as mksd.sh, plus a test SEND.BIN of known pseudo-random bytes.
- `-DSD_PREP_VERIFY` later checks RECV.BIN on board B against the same bytes.

3. SD Card & FAT32 (3/8) FAT32 on disk: from cluster to sector



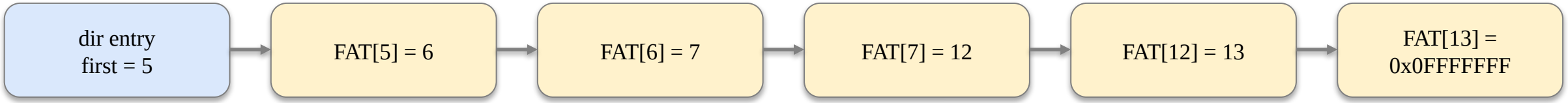

```
fat_lba   = part_lba + reserved                    part_lba = 67584, reserved = 32
data_lba  = fat_lba + nfats * fat_sz               two FAT copies
LBA of cluster c       = data_lba + (c - 2) * spc            spc = sectors per cluster
FAT entry of cluster c = sector fat_lba + c / 128, byte (c % 128) * 4, low 28 bits
                         0 = free,  2 ... = next cluster,  0x0FFFFFFF = end of chain
```

Card (formatted by sd_prep, cluster size as Windows picks it)	spc	Cluster	1 MiB file
board A, 31 GB	32	16 KiB	64 clusters
board B, 3.6 GB	8	4 KiB	256 clusters

All values come from the boot sector at part_lba (bytes 11, 13, 14, 16, 32, 36, 44, 48): fat32_mount() reads them and nothing else.

3. SD Card & FAT32 (4/8) Fragmented files: following the cluster chain

```
root-directory entry, 32 bytes
53 45 4E 44 20 20 20 20 42 49 4E   "SEND    BIN"   name, 8.3 form
20                                 attribute: archive
... bytes 20-21 / 26-27            first cluster, high / low 16 bits
... bytes 28-31                    file size, little endian
```




```
fat32_read()
if (f->pos && f->pos % cb == 0) {        // crossed into the next cluster
    fat_get(f->clus, &f->clus);          // read its FAT entry
    if (!valid(f->clus)) return FAT_ENOFS;
}
```

- Reading follows any chain: a SEND.BIN a PC wrote in pieces works (host test "copy fragmented SEND.BIN"). Cost: one FAT sector per cluster, 63 extra reads for 1 MiB on card A (3 %).
- Writing takes one **contiguous** run of free clusters: no FAT reads while streaming, the chain is written at commit.
- Limit: a card whose free space exists only in pieces answers “no room” (FAT_ENOSPC). Copy the files off and reformat.

3. SD Card & FAT32 (5/8) The SD driver and three bugs

Step	Command
Wake up in SPI mode	≥ 74 clocks, then CMD0 (go idle) — **retried**
Check voltage / version	CMD8 → echo 0x1AA
Initialise	ACMD41 until ready; CMD58 → block addressing (SDHC)
Read / write one sector	CMD17 / CMD24, 512 B + CRC-16, then wait while busy

- **Bug 1:** after power-up the card answers the first CMD0 with one junk byte (Lesson 2) → retry CMD0.
- **Bug 2:** after a write the card holds the data line low while busy. The code waited only while it read exactly 0x00; the line is released in the middle of a byte (e.g. 0x0f), so the next command was sent too early and failed. Fix: wait until it reads **0xFF**.
- **Bug 3:** one card sometimes sends its write reply (data response, 05 = accepted) one byte late. Reading only the next byte saw FF and reported a failed write. Fix (*data_response()*, as Linux mmc_spi does): read 4 bytes and find the reply's start bit; each rejected sector is now retried and logged (`SD: write …`).
- Check bug 1: let CMD0 run once (`t < 1` in *tests/src/periph_spi.c*), pull the card out and in, run spi_test → R1 = 3f FAIL; restore with `git checkout`. Bugs 2 and 3: run several 1 MiB transfers — no `SD:` lines, and the read-back matches (section 4).

3. SD Card & FAT32 (6/8) One sector over SPI

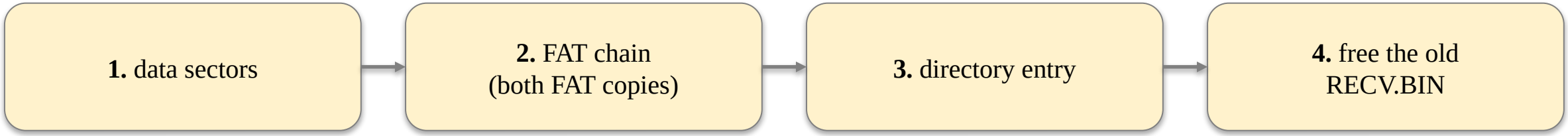
```
reading the FAT32 boot sector of the card
-> 51 00 01 08 00 01          CMD17 read block: 0x40 | 17, LBA 0x00010800 = 67584, CRC byte
<- FF 00                      R1 = 0x00: command accepted
<- FF ... FF FE               start token
<- 512 data bytes, CRC-16     we check the CRC-16 ourselves

-> 58 <LBA, 4 B> 01           CMD24 write block
<- 00                         R1
-> FE, 512 data bytes, CRC-16
<- xxx0 0101 (0x05)           data response: accepted
<- 00 00 ... 0F FF            busy while the card programs: wait for FF (bug 2)
```

- Every command is 6 bytes: 0x40 | index, a 32-bit argument, CRC-7 + stop bit. In SPI mode only CMD0 (95) and CMD8 (87) need a real CRC.
- SDHC / SDXC cards take the sector number; old SDSC cards a byte address (LBA × 512). The CCS bit of CMD58 says which.
- One sector per command: multi-block commands would save little, since the SD part is 5.7 s (read) and 6.5 s (write) of the 99.6 s transfer.

3. SD Card & FAT32 (7/8) FAT32 in 512 bytes of RAM

- One 512-byte sector buffer for everything: no FAT cache, no cluster buffer — the 64 KiB RAM cannot afford more.
- **Read** SEND.BIN: find it in the root directory (8.3 name), follow its cluster chain sector by sector.
- **Write** RECV.BIN: reserve one contiguous run of free clusters, write the data, then **commit** in a safe order:



- Rejected **before** commit (bad record, wrong CRC, lost link): only free clusters were written, so every file — including the old RECV.BIN — is unchanged.
- Commit itself is several sector writes. An I/O error or power loss **during** commit can leave lost clusters or a half-written chain: run *fsck.fat* on a PC. This is not a power-loss-safe file system.

3. SD Card & FAT32 (8/8) Testing the FAT32 code on the PC

```
cd tests && make fat32-host-test   (excerpt of 15 checks)
  copy fragmented SEND.BIN over old RECV.BIN   PASS
  RECV.BIN == SEND.BIN (pyfatfs)               PASS
  fsck.fat clean                               PASS
  uncommitted write leaves files unchanged     PASS
  empty SEND.BIN -> new RECV.BIN               PASS
  1000-byte SEND.BIN -> new RECV.BIN           PASS
  format a junk card                           PASS
  MBR: p1 raw at 2048, p2 FAT32 at 67584       PASS
  copy on the formatted card                   PASS
  missing file -> FAT_ENOENT                   PASS
FAT32 HOST TEST: PASS (0 failures)
```

- Same fat32.c, with sd_read / sd_write going to a disk-image file.
- Images made by *mkfs.vfat*, checked by **fsck.fat** and an independent FAT library (pyfatfs).
- Does the test catch bugs? Two deliberately broken builds (no FAT mirroring, old chain not freed) both **FAIL** fsck.

Outline

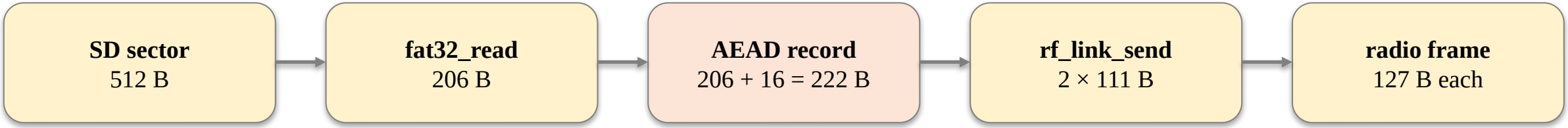
1. Application Channel
2. Radio Link
3. SD Card & FAT32
4. **End-to-End Transfer**
5. Making It Faster
6. Questions Behind the Project
7. Practice

4. End-to-End Transfer (1/8) What goes over the air

Record	Plaintext	Purpose
0	4-byte length (0xFFFFFFFF = sender has no SEND.BIN)	receiver reserves space
1 … n	the file, 206 bytes per record, last one shorter	data
n + 1	4-byte CRC-32 of the file	end-to-end check, card to card

- Every record is AEAD-protected (section 1) — that is the security. The CRC-32 checks the whole chain: SD read → radio → SD write.
- Neither side holds more than one record and one sector: the same image moves 1 KB or 100 MB.
- Receiver busy writing the card? The link just repeats fragments until it reads again — no extra flow control needed.

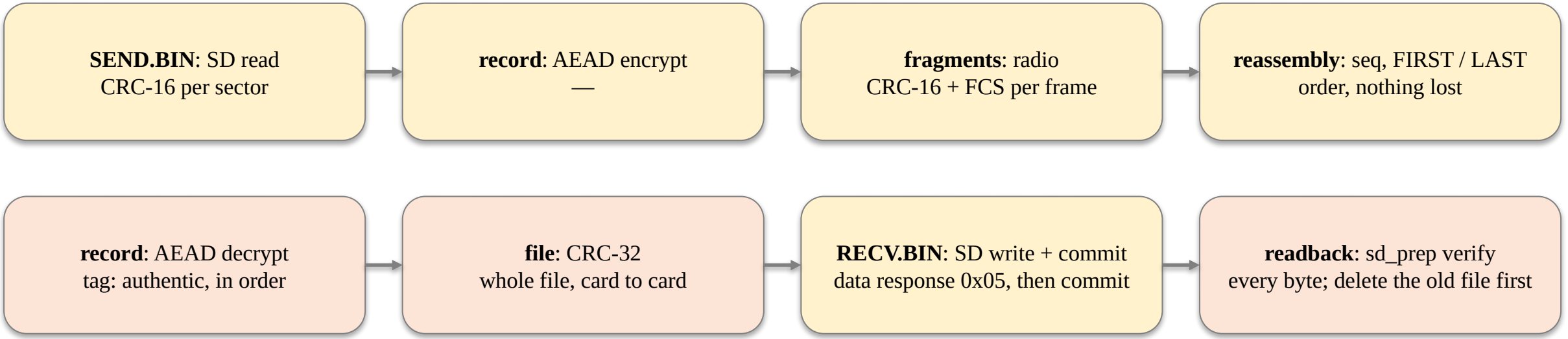
4. End-to-End Transfer (2/8) From file bytes to radio frames



1 MiB, card A → card B	Count
SD sectors read on A / written on B	2,048 / 2,048
records: 5,090 × 206 B + 1 × 36 B, plus the length and CRC records	5,093
data frames ('D'), and as many 'A' frames back	10,183
time per data frame: 99.6 s / 10,183	≈ 9.8 ms

- Records do not line up with sectors (512 = 2 × 206 + 100): fat32_read copies across sector borders, and the one-sector buffer still reads each sector once.
- No buffer grows with the file: one record, a few frames and one sector, the same for 1 KB or 4 GB.

4. End-to-End Transfer (3/8) A check at every step



Console line	Proves	Does not prove
XFER SENT (board A)	A read its file and every record was accepted by the link	anything about B's card
… CRC cffbfb2e OK (board B)	B received exactly the bytes of SEND.BIN	that they were stored
XFER RECV … -> RECV.BIN	B also committed RECV.BIN	that the card holds them correctly
sd_prep: 0 bytes differ	the stored file, read back, is right	—

4. End-to-End Transfer (4/8) Build and run

```
Run this
cd fpga/sw/export/edhoc-authkem
make edhoc_authkem_initiator edhoc_authkem_responder SUITE=TBD3 EXTRA_CFLAGS=-DAPP_SD_XFER
../common/board/run_pair.sh xfer build-tbd3/edhoc_authkem_initiator.elf \
                                build-tbd3/edhoc_authkem_responder.elf 1500
# then check board B's card:
cd ../tests && make sd_prep EXTRA_CFLAGS=-DSD_PREP_VERIFY
BOARD=responder ../common/board/run_one.sh build/sd_prep.elf 20
```

- Works with every suite of Lessons 3 and 4 (`edhoc-wolfssl` takes the same `-DAPP_SD_XFER`).
- Board A needs SEND.BIN on its card; board B needs a FAT32 partition.

4. End-to-End Transfer (5/8) Real run: 1 MiB from card A to card B

```
board A (initiator)
XFER: sending SEND.BIN, 1048576 bytes
PERF xfer_send    7465350439 cyc 99.538005 s 1048576 bytes 10534 B/s
PERF xfer_sd_part      427826394 cyc 5.704351 s
XFER SENT: 1048576 bytes, CRC cffbfb2e
```

```
board B (responder)
XFER: receiving 1048576 bytes
PERF xfer_recv    7469254102 cyc 99.590054 s 1048576 bytes 10528 B/s
PERF xfer_sd_part      490423540 cyc 6.538980 s
XFER RECV: 1048576 bytes -> RECV.BIN, CRC cffbfb2e OK
```

```
board B, sd_prep verify
card: 7569408 sectors (3696 MiB)
RECV.BIN: 1048576 bytes, CRC cffbfb2e, 0 bytes differ
SD PREP OK
```

- **1 MiB in 99.6 s = 10.5 KB/s**, post-quantum TBD3 keys, AES-256-GCM.
- SD time: 5.7 s read + 6.5 s write — the radio sets the pace.

4. End-to-End Transfer (6/8) When something is missing

```
board A, card without FAT32
XFER: SD card not usable (-2)
XFER FAIL 3
```

```
board B
XFER: peer has no SEND.BIN
XFER FAIL 3
```

- The sender announces “no file” (length 0xFFFFFFFF) instead of going silent: both boards stop cleanly in 3 s, no 5-minute timeout.
- If board B's card fails, it still receives and checks the whole stream (CRC) and reports “not stored” — both sides stay in step.
- A wrong CRC or a broken record: RECV.BIN is not written, the old one stays.

4. End-to-End Transfer (7/8) When storage fails: a real run

```
Example output — board A
XFER: sending SEND.BIN, 1048576 bytes
XFER SENT: 1048576 bytes, CRC cffbfb2e
APP DATA OK
```

```
Example output — board B, same run (before the bug 3 fix)
XFER: receiving 1048576 bytes
XFER: SD write failed at 618 (-1)
XFER: received 1048576 bytes, CRC cffbfb2e OK, not stored (-1)
XFER FAIL 3
APP DATA FAIL: 3
```

```
Example output — run_pair.sh
xferfix2 FAIL hs=1.236142s ... 122s no-APP-OK(xferfix2.res.log)
```

- The communication path worked: the handshake, every record and the file CRC are fine. **Storage failed**: the first sector write (offset 618 = 3 records, past 512 B) was reported as refused, so B kept receiving and checking but did not store.
- Board A's “XFER SENT … APP DATA OK” is true and still not success. Judge a transfer by board B's last lines — and then read the file back (sd_prep verify).
- Cause, found by logging the card's reply: its data response came one byte late (section 3, bug 3). Current firmware prints `SD: write <sector>: <reason>` for every rejected try — read those lines before blaming the card.
- Read-back trap: rename or delete the old RECV.BIN before a run. Otherwise a failed run leaves the previous, identical file in place and the read-back still says 0 bytes differ.

4. End-to-End Transfer (8/8) Larger files: how to run the experiment

Step	How	Watch out
1. Make the file	copy it as SEND.BIN with a card reader, or sd_prep -DSD_PREP_SEND=<bytes>	FAT32: up to 4 GiB per file
2. Room on card B	free space ≥ file size, in **one contiguous run** (section 3)	else “no room” (FAT_ENOSPC): reformat B
3. Timeout	4th argument of run_pair.sh: size / 10 KB/s + 60 s	1 MiB ≈ 100 s → 200 s is safe
4. Run, keep logs	LOGDIR=xfer_runs ../common/board/run_pair.sh …	one log pair per run
5. Verify	sd_prep -DSD_PREP_VERIFY on board B (or sha256sum on a PC)	0 bytes differ
6. Compare	PERF xfer_send / xfer_recv B/s and the STACK lines per size	RAM use should not grow

- Units here: 1 MiB = 1,048,576 B; KB/s = 1,000 B/s; PERF prints B/s. Measured: 1 MiB in 99.6 s. Anything larger in these slides is an **estimate** from that rate until you measure it.

Outline

1. Application Channel
2. Radio Link
3. SD Card & FAT32
4. End-to-End Transfer
5. **Making It Faster**
6. Questions Behind the Project
7. Practice

5. Making It Faster (1/7) Measure first

Per radio frame (sender, 111 B of data)	Time
radio TX + hardware ACK	7.0 ms
waiting for our 'A'	1.75 ms
AES-256-GCM encryption (before tuning)	3.1 ms

- Cycle counters around each step, in a temporary instrumented build — not guesses. Numbers on this slide come from that build, not from TIMING=1.
- Encrypting one 206-byte record took **6.5 ms = 490,000 cycles**. That is far too slow for AES: something was wrong.
- On-board micro-benchmark:

AES-256 block	GHASH multiply	Key schedule	GCM, 192 B
14,123 cyc	19,777 cyc	6,231 cyc	490,695 cyc

5. Making It Faster (2/7) The multiply trap

```
before (GHASH, aes_gcm.c)
uint64_t m = 0 - (bit & 1);      // all ones if bit is set
zh ^= vh & m;                    // GCC turns this into  bit * vh  -> mul
```

```
after
static inline uint64_t top_mask(uint64_t x) { return (uint64_t)((int64_t)x >> 63); }
uint64_t m = top_mask(xh);       // arithmetic shift: no multiply
zh ^= vh & m;
```

- GCC recognises `(0 - bit) & v` and emits `bit * v` — usually faster. But this core's multiplier is **bit-serial: ~64 cycles per mul**.
- Two muls per bit × 128 bits per GHASH block: 128 of every 155 cycles were multiplies.
- Result: GHASH **19,777 → 3,326 cycles** (6×). Same trap fixed in the CRC-32 code. Check with `objdump -d … | grep mul`.

5. Making It Faster (3/7) Table-based AES

- The byte-by-byte AES rounds became the 32-bit **T-table** form: each round is 16 table look-ups and XORs on whole columns.
- One 1 KB table, built at first use from the S-box; the other three are its byte rotations.

Cycles	Before	After	Speed-up
AES-256 block	14,123	2,224	6.3×
GHASH multiply	19,777	3,326	5.9×
AES-256-GCM, 192-byte record	490,695	91,741	5.3×

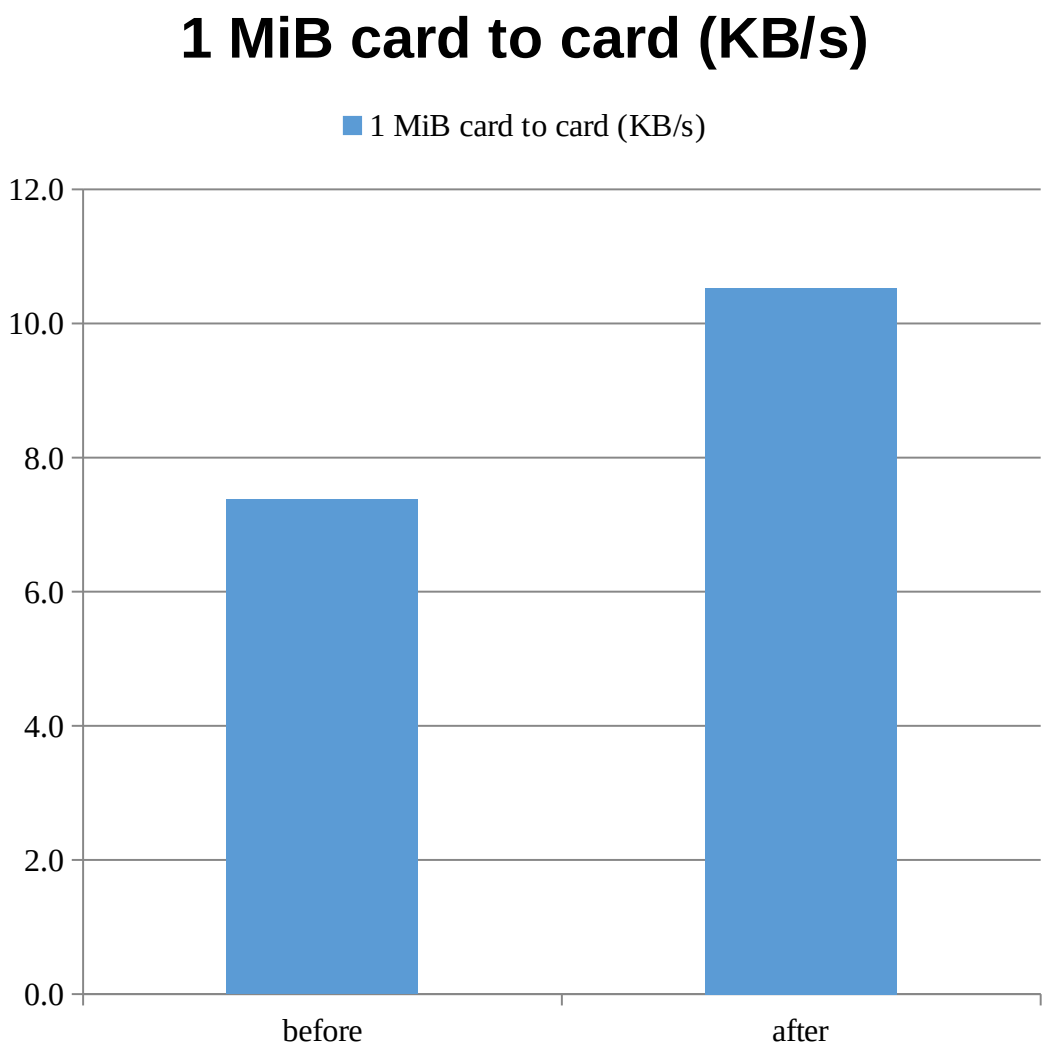
- All known-answer tests (NIST GCM, RFC 3610, RFC 9529) still PASS. Bulk speed on the board now: encrypt **162 KB/s**, decrypt **153 KB/s**.
- Cost: 1 KB of RAM — the largest builds still fit.

5. Making It Faster (4/7) The radio side

Change	Why	Kept?
Burst SPI: whole FIFO in one command	1 SPI byte per data byte instead of 3	yes
No CSMA back-off before sending	two nodes taking turns: no one to avoid	yes
206-byte records	fill both radio fragments	yes
Turbo mode 625 kbit/s	~2/3 of frames lost on these modules	**no**
Channel 26 (above Wi-Fi)	much worse than channel 11 here	**no**
SPI at 9.4 MHz	byte errors in the burst test	**no**

- Each change tested on the boards; channel 11 has interference spells, so every experiment was compared with a control run made at the same time.

5. Making It Faster (5/7) Result



1 MiB, card A → card B, TBD3	Before	After
Time	142 s	99.6 s
Throughput	7.37 KB/s	10.53 KB/s
Bytes wrong	0	0
100 MiB (estimate, not measured)	≈ 4 h	≈ 2.8 h

- **+43 %**: 104 s after these changes, 99.6 s with the link fix of section 2 (Problem 3). Handshakes got a little faster too (TBD3 1.37 → 1.25 s).
- Check yourself: a TIMING=1 build prints PERF encrypt / decrypt in B/s; compare with the micro-benchmark of (1/7).

5. Making It Faster (6/7) Where the limit is now

Raw radio, no ACK	Time per frame
full frame (116 B payload)	6.3 ms
tiny frame (5 B payload)	2.5 ms

- From the slope: 32 µs per byte (250 kbit/s) plus **≈ 1.8 ms fixed per frame inside the radio chip**.
- Software is now ~1 ms of the ~9 ms per frame. The radio chip sets the speed.
- A sliding window could save the ~19 % spent waiting for 'A' — risky with a one-frame receive buffer (Problem 3 in section 2 shows why).
- Faster needs a faster link: three wires between the PMOD JB ports give 3× the radio's speed (next slide).

5. Making It Faster (7/7) A wired link: UART1 at 921,600 baud

TBD3, measured on the boards	Radio (MRF24J40)	UART, 57,600 baud, byte pacing	UART, 921,600 baud, flow control
Handshake	1.25 s	3.07 s	0.77 s
256 KiB generated data	—	2.4 KB/s	49.5 KB/s
1 MiB, SD card A → SD card B	99.6 s (10.5 KB/s)	—	32.2 s (32.6 KB/s)

- Wiring: A's JB2 (TX) → B's JB1 (RX), B's JB2 (TX) → A's JB1 (RX), JB5 GND ↔ JB5 GND (Lesson 1, section 4). Build both images with `TRANSPORT=uart`; `UART1_BAUD=<baud>` changes the speed.
- The receiver polls UART1 and is busy between frames (AEAD, SD writes), so a sender that just transmits loses bytes. The first version waited after **every byte**.
- Now the receiver sends one READY byte every 0.2 ms while it waits for a frame; the sender drops old READYs, waits for a fresh one, then sends the frame at line rate (*edhoc_transport.c*). With that, the baud rate sets the pace.
- Card to card, AES and the SD card now take a larger share: 5.7 s of the 32.2 s is SD access on each board.

Outline

1. Application Channel
2. Radio Link
3. SD Card & FAT32
4. End-to-End Transfer
5. Making It Faster
6. **Questions Behind the Project**
7. Practice

6. Questions Behind the Project (1/3) The questions, and where they are answered

Question	Short answer	Lesson
Does EDHOC fit a small Rocket SoC with 32 KiB?	Only method 3 of suites 0/1/4; the rest need up to 61 KB → 64 KiB	1, 2
Why not 100 MHz?	this implementation failed timing at 100 MHz and closes at 75 MHz	1
Do all methods and suites work?	methods 0–3 × 9 suites run on two boards; the two RFC 9529 traces match byte for byte	3
Does post-quantum EDHOC need a PSK?	no shared PSK; still trusted credentials and long-term ML-KEM private keys	4
Is there a highest-security option?	TBD3, the strongest configured here: AES-256-GCM + ML-KEM-1024 + SHA-384	4
Where do large files live, how much RAM?	FAT32 on the SD card, streamed: RAM use is flat	5
How long does each step take?	TIMING=1 prints PERF lines per step	3, 4, 5

The questions were asked during the project; *deviations.txt* keeps the dated questions and answers.

6. Questions Behind the Project (2/3) Limits to know before you reuse this code

Limit	**Why**	**What to do**
Random numbers (RFC 9528 §9.2)	this SoC has no TRNG; demo seeds come from the cycle counter	connect edhoc_platform_entropy() to real entropy
Post-quantum drafts	TBD1–3 unassigned; draft -01 may change	labels: one define; a new revision: full review + reference model
Radio speed	250 kbit/s chip, ~10.5 KB/s end to end; 100 MiB ≈ 2.8 h (estimate)	TRANSPORT=uart over 3 wires on PMOD JB: 32.6 KB/s card to card
SD storage	commit is not power-loss safe; RECV.BIN needs contiguous free space	verify by readback; fsck.fat after a crash
32 KiB chips	only method 3 of suites 0/1/4 fits	use those builds, or give the chip more RAM
Boot from SD after power-up	the boot ROM sends CMD0 only once	load over JTAG (Lesson 2), or add a CMD0 retry to sdboot and rebuild the bitstream

6. Questions Behind the Project (3/3) Reproduce everything

Step	Lesson	You should see
1. Clone (Lesson 1, (4/7)), build the bitstream, program both boards	1	timing met, jtag_test PASS
2. Toolchain, peripheral + crypto tests	2	every test PASS
3. EDHOC pairs, RFC 9529 test	3	PASS per pair, RFC9529 PASS
4. AuthKEM TBD1–3, reference model	4	PASS (prk=match with DEBUG=1), REFERENCE MODEL: PASS
5. Prepare cards, transfer, verify RECV.BIN	5	XFER RECV … OK, then 0 bytes differ

- Every number in these lessons comes from such runs; the log format (PERF / STACK lines) and the scripts (run_pair.sh, perf_table.py) let you compare yours line by line.
- `VERBOSE=1 run_pair.sh ...` shows both consoles live; a DEBUG=1 build adds every intermediate value.

Outline

1. Application Channel
2. Radio Link
3. SD Card & FAT32
4. End-to-End Transfer
5. Making It Faster
6. Questions Behind the Project
7. **Practice**

7. Practice (1/3) Exercises

Exercise 1:

- Send a real file (a PDF, an image) from card A to card B with TBD3. Compare the two files on a PC (sha256sum).

Exercise 2:

- Build the radio link without the CRC (remove `seal` and the check). Stream 1 MiB. What fails, after how long? Restore the file afterwards.

Exercise 3:

- Measure KB/s for 64 KiB, 256 KiB and 1 MiB. Is the speed constant? Estimate the time for 100 MB.

7. Practice (2/3) What to put in the report

- Exercise 1: both consoles, the two sha256 values, the time and KB/s.
- Exercise 2: the console at the failure and your explanation.
- Exercise 3: a table and a chart of size vs. time; your 100 MB estimate.

7. Practice (3/3) Answer notes

Exercise	What you should see	Why
1. Real file	identical sha256 on both files; ≈ 100 s per MiB plus a few seconds of handshake	CRC-32 and AEAD protect the path; the radio sets the pace
2. No CRC	sooner or later an AEAD failure on B (record at <n> failed (113)) — or, on a lucky run, none	a flipped SPI bit is no longer caught before the AEAD sees it
3. Sizes	about the same B/s for every size	the cost is per frame; nothing in RAM grows with the file

- Restore after exercise 2: from *fpga/sw/export*, `git checkout common/src/edhoc_rf_link.c`, rebuild both images, rerun: XFER RECV … OK.
- Negative cases to recognise: no SEND.BIN → both stop with XFER FAIL 3 in seconds; B's card fails → B says “not stored” while A says SENT (section 4, (7/8)); a broken record → B rejects it and keeps the old RECV.BIN.

Today's lesson highlight:

- PRK_out → EDHOC_Exporter → K_app / IV_app; 206-byte AEAD records, nonce from direction + sequence number.
- Radio link made reliable with a CRC per frame, a 2-second radio watchdog and reading the FIFO while repeating; SD driver fixed (CMD0 retry, busy wait, late write reply).
- FAT32 in one 512-byte buffer: SEND.BIN → RECV.BIN, committed only when the CRC matches.
- 1 MiB card to card with TBD3: 99.6 s, 0 bytes wrong; +43 % after removing hidden multiplies, table AES and the link fix.
- A transfer succeeded only when board B says RECV … OK **and** the read-back matches — the sender's "SENT" is not enough.
- Before real use: a real entropy source, and final post-quantum code points.

End of the course

Thank you for following all five lessons

THANK YOU

2026/12